SHIFT HAPPENS

Guía técnica de movimiento de NPCs

Navegación 2D, destinos, reservas, conexiones entre plantas e integración con la futura API de estados.

PROYECTO	MOTOR	PLATAFORMA OBJETIVO
Shift Happens	Unity 6 · 2D	WebGL
VERSIÓN DOCUMENTAL	**FECHA**	**ÁMBITO**
1.0	8 noviembre 2026	Programación

Documento de referencia para el equipo de desarrollo. Describe el funcionamiento de los scripts actuales y cómo configurarlos en Unity. La API de máquinas de estados todavía no está definida en esta guía: se explica el contrato de integración sin asumir sus clases o métodos concretos.

1. Alcance y arquitectura

Esta guía documenta la implementación actual del movimiento autónomo de los NPCs de Shift Happens. El objetivo es que cualquier programador del equipo pueda crear nuevos puntos de interés, conectar plantas y solicitar desplazamientos sin conocer los detalles internos del NavMesh.

Idea principal

La máquina de estados decide qué actividad quiere realizar el NPC. El sistema de navegación resuelve cómo llegar al lugar y avisa cuando termina el desplazamiento. No se debe duplicar la lógica del NavMesh dentro de la IA.

1.1. Componentes y responsabilidades

Componente	Responsabilidad
NPCDestination	Representa un lugar de actividad: nombre, tipo, planta y posiciones de ocupación (slots). Gestiona las reservas.
NPCDestinationManager	Busca destinos del tipo y planta solicitados, filtra los que tienen capacidad y prueba candidatos en orden aleatorio.
NPCFloorConnection	Representa una conexión entre dos plantas mediante un punto de entrada y otro de salida. Puede ser bidireccional.
NPCNavigation	Controla el NavMeshAgent dentro de una superficie: calcula rutas, mueve, detiene, comprueba accesibilidad y hace Warp.
NPCFloorNavigator	Es la puerta de entrada para el movimiento completo: reserva un slot, calcula la secuencia de plantas con BFS, recorre escaleras y emite eventos.
NPCFloorNavigationTest	Controlador temporal de teclado para probar peticiones de destinos. No forma parte de la IA definitiva.

1.2. Flujo general de una petición

1 · IA / estado	Decide la actividad: por ejemplo, ir al baño de la primera planta.
2 · DestinationManager	Busca destinos de tipo Bathroom en FirstFloor y descarta los que no tienen slot libre.
3 · FloorNavigator	Reserva un slot concreto, valida el recorrido y decide si necesita escaleras.
4 · Navigation	Sigue la ruta local con NavMeshAgent. En cada transición se ejecuta Warp al otro extremo.
5 · Evento de llegada	OnDestinationReached informa a la IA. El slot sigue reservado hasta liberar la actividad.

1.3. Plantas del edificio

El edificio tiene cuatro niveles navegables. Las conexiones son lineales y bidireccionales:

Sótano ↔ Planta baja ↔ Primera planta ↔ Segunda planta

En código corresponden a Basement, GroundFloor, FirstFloor y SecondFloor. No hay desván en el diseño actual.

2. Preparación de la escena en Unity

El proyecto utiliza un escenario 2D cenital en el plano XY. Para la navegación se utiliza NavMeshPlus sobre el sistema de NavMesh de Unity.

2.1. Paquetes y componentes

- Instalar AI Navigation desde Package Manager y la extensión NavMeshPlus desde su repositorio Git.
- En el objeto de navegación de la escena utilizar el Navigation Surface y el Navigation CollectSources2d de NavMeshPlus.
- Rotar la superficie para generar el NavMesh en XY. Configurar la geometría 2D a partir de colliders y los modificadores de zonas transitables/no transitables según la configuración de la escena.
- Los agentes de los NPCs llevan NavMeshAgent, NPCNavigation y NPCFloorNavigator. El agente debe utilizar el mismo Agent Type que el NavMesh horneado.

2.2. Jerarquía recomendada

```
Scene
├── Floor_Basement
├── Floor_Ground
├── Floor_First
├── Floor_Second
├── Navigation
├── Destinations
│   ├── Destination_Bathroom_Ground
│   ├── Destination_Kitchen_Ground
│   └── ...
├── FloorConnections
│   ├── Connection_Basement_Ground
│   ├── Connection_Ground_First
│   └── Connection_First_Second
└── NPCs
    ├── NPC_Student_01
    └── NPC_Student_02
```

La jerarquía es orientativa. Lo importante es mantener separados los objetos que definen las plantas, los destinos, las conexiones y los NPCs. Cada NPC referencia las conexiones que puede utilizar; el DestinationManager contiene las ubicaciones disponibles.

2.3. NavMesh y mapa de una sola escena

La escena puede contener las cuatro plantas a la vez. Para el prototipo se han colocado las plantas como regiones separadas del espacio XY; cada región tiene su propio suelo y obstáculos, mientras una superficie de navegación recoge las zonas transitables. BFS no crea la ruta física entre plantas: la ruta se construye a partir de las conexiones y el NPC se teletransporta de un NavMesh local al siguiente.

> **Importante al montar el mapa definitivo**
>
> Las posiciones de los puntos de escaleras, salidas y slots deben caer sobre la zona azul del NavMesh correspondiente. Si las cuatro plantas se dibujan exactamente superpuestas en las mismas coordenadas XY, la geometría de navegación puede solaparse y el agente no podrá distinguir qué planta representa una posición. Mantener las regiones separadas en XY (como en la escena de prueba) o acordar una estrategia de NavMesh por planta antes de solapar geometría.

2.4. WebGL

El NavMesh debe hornearse en el Editor y probarse dentro de una compilación WebGL real. No se debe dar por validada la compatibilidad por el hecho de que funcione en Play Mode. La prueba final debe incluir movimiento local, Warp entre plantas, reservas y varios agentes en el navegador.

3. Crear puntos de destino para los NPCs

Un destino es un lugar lógico donde un NPC puede realizar una actividad. No es necesariamente un objeto interactivo del jugador: puede representar un baño, una zona de estudio, una cocina o un lugar donde esperar.

3.1. Tipos y plantas disponibles

Enum	Valores actuales
BuildingFloor	Basement · GroundFloor · FirstFloor · SecondFloor
NPCDestinationType	Generic · Bedroom · Bathroom · Kitchen · StudyRoom · CommonRoom · Reception · Laundry

3.2. Crear un destino paso a paso

1. En la Hierarchy, dentro de Destinations, crear un GameObject vacío. Darle un nombre descriptivo, por ejemplo Destination_Bathroom_First.

2. Colocarlo dentro de la zona transitable de la planta correcta. La posición raíz sirve como slot por defecto cuando no se configuran slots hijos.

3. Añadir el componente NPCDestination.

4. Rellenar Display Name, Destination Type y Floor en el Inspector.

5. Añadir el GameObject al array Destinations del NPCDestinationManager. Si no se registra en ese array, el gestor no podrá seleccionarlo.

3.3. Qué hace el destino si no se crean slots

Si el array Slots está vacío, NPCDestination utiliza su propio Transform como una única posición de ocupación. Esto es apropiado para un lugar individual, como un baño de una sola plaza o una habitación concreta.

3.4. Crear slots para una zona con varias plazas

1. Dentro del GameObject destino, crear objetos vacíos hijos: Slot_01, Slot_02, etc.

2. Colocar cada Transform donde deba terminar un NPC: por ejemplo, delante de un escritorio, junto a una encimera o en una posición de la sala común.

3. En el componente NPCDestination, asignar todos los hijos al array Slots. La capacidad del destino es el número de slots configurados.

4. Comprobar que ningún slot está vacío y que cada posición se encuentra sobre el NavMesh de esa planta.

> **Cómo se ve en Scene**
> El gizmo cian marca el Transform principal del destino. Los gizmos verdes marcan los slots configurados. Cada NPC reserva una posición verde distinta; no se reserva solo el nombre lógico de la habitación.

3.5. Qué significa la capacidad

En la implementación actual, Capacity equivale al número de slots. Si Slots está vacío, Capacity es 1. CurrentOccupants cuenta las reservas activas, no solo los NPCs que ya han llegado físicamente. Una plaza reservada mientras el NPC camina ya cuenta como ocupada. Dentro de cada destino se asigna el primer slot libre siguiendo el orden del array; el gestor elige el destino de forma aleatoria, pero no sortea el slot.

4. Registro de destinos y conexiones entre plantas

4.1. NPCDestinationManager

El objeto que contiene NPCDestinationManager mantiene el array global de destinos que pueden solicitar los NPCs. Cuando recibe un tipo y una planta, recorre la lista, filtra por coincidencia y disponibilidad y prueba candidatos en orden aleatorio. El intento real de reserva y navegación lo delega en NPCFloorNavigator.MoveToDestination(). Si un candidato falla, prueba el siguiente.

Ejemplo de configuración:

Destino	Tipo	Planta	Slots
Baño planta baja	Bathroom	GroundFloor	1
Cocina	Kitchen	GroundFloor	2
Sala de estudio	StudyRoom	GroundFloor	4
Lavandería	Laundry	Basement	1
Baño primera	Bathroom	FirstFloor	1
Baño segunda	Bathroom	SecondFloor	1

Los valores son un ejemplo de organización; las ubicaciones y capacidades definitivas dependen del diseño del mapa.

4.2. Crear una conexión de escaleras

1. Dentro de FloorConnections, crear un GameObject vacío con nombre descriptivo, por ejemplo Connection_Ground_First.

2. Crear dos hijos vacíos: uno para la entrada/salida de la planta A y otro para la planta B. Los nombres pueden ser Point_Ground y Point_First.

3. Colocar cada punto sobre el NavMesh de su planta. El punto de entrada debe quedar donde el NPC pueda llegar caminando; el punto de salida debe quedar donde pueda continuar caminando después del teletransporte.

4. Añadir NPCFloorConnection al GameObject padre y asignar Floor A, Point A, Floor B, Point B. Activar Bidirectional para permitir subir y bajar.

5. Añadir esa conexión al array Available Connections del NPCFloorNavigator de cada NPC que pueda usarla.

4.3. Conexiones que utiliza Shift Happens

GameObject de conexión	Planta A	Planta B	Bidireccional
Connection_Basement_Ground	Basement	GroundFloor	Sí
Connection_Ground_First	GroundFloor	FirstFloor	Sí
Connection_First_Second	FirstFloor	SecondFloor	Sí

El grafo actual es lineal. BFS encuentra la secuencia de conexiones que une la planta actual con la planta objetivo: por ejemplo, Basement → GroundFloor → FirstFloor → SecondFloor. No hay una conexión a un desván.

Puntos de escalera y radio de muestreo

NPCNavigation utiliza un radio de muestreo de destino de 0,5 unidades para localizar puntos válidos del NavMesh. Colocar cada punto de conexión y cada slot sobre la superficie azul y muy cerca de ella; no situarlos fuera de pasillos ni sobre paredes. El Warp también comprueba el NavMesh cerca del punto de salida.

5. Qué ocurre cuando se mueve un NPC

La petición normal parte de un tipo de actividad y una planta objetivo. El gestor escoge una ubicación; el navegador reserva una posición; y Navigation ejecuta el movimiento local. Si el destino está en otra planta, FloorNavigator recorre las conexiones necesarias.

Puntos de escalera y radio de muestreo

NPCNavigation utiliza un radio de muestreo de destino de 0,5 unidades para localizar puntos válidos del NavMesh. Colocar cada punto de conexión y cada slot sobre la superficie azul y muy cerca de ella; no situarlos fuera de pasillos ni sobre paredes.

5.1. Secuencia completa

Fase	Comportamiento
A. Solicitud	La IA solicita, por ejemplo, Bathroom en SecondFloor.
B. Selección	NPCDestinationManager busca destinos Bathroom/SecondFloor con al menos un slot libre.
C. Reserva	NPCFloorNavigator reserva el slot para ese GameObject y guarda CurrentDestination y CurrentDestinationSlot.
D. Planificación	Si el destino está en otra planta, BFS construye una lista ordenada de conexiones. CanTraverseRoute comprueba cada tramo local hasta una escalera y desde cada salida hasta la siguiente escalera o el slot final.
E. Movimiento	NPCNavigation calcula y sigue una ruta completa en el NavMesh de la planta actual.
F. Transición	Al llegar al punto de entrada, se ejecuta WarpTo al punto de salida; se actualiza CurrentFloor y se emite OnFloorChanged.
G. Continuación	El NPC camina hasta la siguiente escalera o, en la planta final, hasta el slot reservado.
H. Llegada	OnDestinationReached se emite al llegar al slot final. La reserva sigue activa mientras realiza la actividad.

5.2. Cómo funciona BFS en este edificio

BFS (búsqueda en anchura) recorre el grafo de plantas y encuentra una secuencia con el menor número de conexiones. En un edificio lineal con tres conexiones, la ruta es única; por eso BFS es suficiente y no necesitamos introducir Dijkstra u otro algoritmo más complejo para el diseño actual.

5.3. Qué hace NPCNavigation

- CanReach(destination): comprueba una ruta completa desde la posición actual, sin iniciar movimiento.
- CanReachFrom(start, destination): valida un tramo entre dos puntos, aunque el agente esté en otra planta; utiliza NavMesh.CalculatePath y el tipo/áreas del agente.
- MoveTo(destination): valida la ruta y la asigna al agente con SetPath.
- StopMoving(): detiene el agente y borra la ruta actual.
- WarpTo(destination): localiza una posición válida cercana y teletransporta al agente usando NavMeshAgent.Warp.

5.4. Evitación entre agentes

La separación entre NPCs durante el desplazamiento depende de la configuración del NavMeshAgent (Radius, Obstacle Avoidance Type y Priority). En las pruebas del prototipo, dos agentes han podido cruzarse. Las reservas de slots y la evitación local resuelven problemas distintos: reservar un slot evita compartir el mismo punto de actividad; la evitación local ayuda a que los agentes no se atraviesen durante el trayecto.

6. Integración con la futura API de estados

> **No asumir una API todavía**
> La API que se utilizará en clase aún no está documentada en este proyecto. Por tanto, los ejemplos siguientes describen cómo debe comunicarse un estado con el navegador, pero no presuponen nombres de clases, interfaces, callbacks ni métodos de la API externa.

6.1. Responsabilidades: IA frente a navegación

Sistema de estados / IA	Sistema de movimiento
Decide que el estudiante quiere ir al baño.	Busca un destino Bathroom en la planta solicitada.
Decide cuándo empezar o terminar una actividad.	Reserva un slot y lleva al NPC hasta él.
Decide qué hacer si no hay destino disponible o falla la petición.	Devuelve éxito/fallo y emite eventos de movimiento.
Al terminar la actividad, ordena liberar el lugar.	Libera el slot reservado.
Si la actividad se interrumpe, ordena cancelar el viaje.	Detiene el agente y libera la reserva.

6.2. Solicitar un destino desde un estado

El punto de entrada sencillo para una IA es NPCDestinationManager.TryRequestRandomDestination. Recibe el tipo, la planta, el NPCFloorNavigator del personaje y devuelve el destino elegido si se ha podido iniciar el movimiento.

```
bool accepted = destinationManager.TryRequestRandomDestination(
    NPCDestinationType.Bathroom,
    BuildingFloor.FirstFloor,
    floorNavigator,
    out NPCDestination selectedDestination
);

if (!accepted)
{
    // La IA decide si espera, cambia de actividad o vuelve a intentarlo.
}
```

Si la IA ya conoce un destino concreto, puede llamar directamente a floorNavigator.MoveToDestination(destination). En ambos casos, las decisiones de ruta, slots, escaleras y teletransporte permanecen dentro del sistema de navegación.

6.3. Suscribirse a los eventos

Un componente adaptador o el estado correspondiente puede escuchar eventos de NPCFloorNavigator. Registrar las suscripciones al activarse y retirarlas al desactivarse evita callbacks duplicados.

```
private void OnEnable()
{
    floorNavigator.OnMovementStarted += HandleMovementStarted;
    floorNavigator.OnDestinationReached += HandleDestinationReached;
    floorNavigator.OnMovementFailed += HandleMovementFailed;
    floorNavigator.OnMovementCancelled += HandleMovementCancelled;
    floorNavigator.OnFloorChanged += HandleFloorChanged;
}

private void OnDisable()
{
    floorNavigator.OnMovementStarted -= HandleMovementStarted;
    floorNavigator.OnDestinationReached -= HandleDestinationReached;
    floorNavigator.OnMovementFailed -= HandleMovementFailed;
    floorNavigator.OnMovementCancelled -= HandleMovementCancelled;
    floorNavigator.OnFloorChanged -= HandleFloorChanged;
}
```

Los manejadores son métodos del componente/estado que consuma los eventos. No deben duplicarse las suscripciones cada vez que se solicite una ruta.

7. Contrato de integración y buenas prácticas

7.1. Eventos y propiedades disponibles

Miembro	Cuándo usarlo
IsMoving	Consultar si el NPC tiene movimiento activo o está recorriendo una secuencia de plantas.
CurrentFloor	Consultar la planta que el navegador considera actual.
CurrentDestination	Consultar el destino lógico reservado.
CurrentDestinationSlot	Consultar el Transform concreto reservado dentro del destino.
OnMovementStarted(destination)	Se emite cuando se acepta una nueva orden de movimiento.
OnDestinationReached(destination)	Se emite cuando el agente llega al slot final. No libera la reserva.
OnMovementFailed(destination)	Se emite cuando falla una petición o un teletransporte gestionado por FailCurrentTrip.
OnMovementCancelled(destination)	Se emite al cancelar un viaje que estaba en movimiento y tenía un destino.
OnFloorChanged(previous, current)	Se emite después de un Warp de planta completado correctamente.

7.2. Terminar una actividad y liberar el slot

Llegar al destino no libera el slot. El NPC mantiene reservada su posición mientras realiza la actividad. Cuando el estado de IA decida que ha terminado, debe llamar:

```
floorNavigator.ReleaseCurrentDestination();
```

Si debe interrumpir un desplazamiento que sigue en curso, debe llamar:

```
floorNavigator.CancelCurrentTrip();
```

Al desactivarse el NPC, NPCFloorNavigator también cancela el viaje y libera el destino desde OnDisable. NPCDestination contiene una limpieza adicional de referencias nulas cuando consulta las reservas.

7.3. Diferencia entre fallo y cancelación

La IA debería distinguir entre una cancelación intencionada (OnMovementCancelled) y un fallo de navegación (OnMovementFailed). En la versión actual, algunos fallos durante una transición intermedia llaman a CancelCurrentTrip; por tanto, esos casos pueden notificarse como cancelación y no como fallo. Conviene unificar ese comportamiento antes de depender de él en decisiones críticas de la máquina de estados.

7.4. Controlador temporal de pruebas

Tecla	Petición actual
1	Bathroom · GroundFloor
2	StudyRoom · GroundFloor
3	Kitchen · GroundFloor
4	Bathroom · FirstFloor
5	Laundry · Basement
6	Bathroom · SecondFloor
C	CancelCurrentTrip()

Para un segundo NPC puede activarse Use Numpad Controls. En el script compartido, la comprobación de la tecla de cocina del numpad contiene una errata: usa Keyboard.current.kKey; debe cambiarse a Keyboard.current.numpad3Key. El componente NPCNavigationTest antiguo (control por clic) debe permanecer desactivado en NPCs que reciban órdenes a través de NPCFloorNavigator.

8. Configuración de un NPC y lista de comprobación

8.1. Componentes del GameObject NPC

Componente	Configuración / referencia
SpriteRenderer	Sprite del personaje. El orden de dibujado (Sorting Layer / Order in Layer) es visual y no sustituye la navegación.
NavMeshAgent	Agent Type compatible con el NavMesh; Radius, Speed, Acceleration, Stopping Distance y Obstacle Avoidance ajustados al tamaño del NPC.
NPCNavigation	Control local del NavMeshAgent. Ajusta updateRotation y updateUpAxis para el plano 2D.
NPCFloorNavigator	Current Floor correcto y array Available Connections con las tres conexiones del edificio.
NPCFloorNavigationTest	Solo para pruebas. Referencias a NPCFloorNavigator y NPCDestinationManager; retirar/desactivar para el build final.

8.2. Lista de comprobación al añadir un destino

- ☐ El GameObject tiene NPCDestination y nombre descriptivo.
- ☐ Display Name, Type y Floor corresponden a la ubicación real.
- ☐ El destino y cada slot están sobre el NavMesh de la planta correcta.
- ☐ Si hay varias posiciones, todos los elementos del array Slots están asignados y separados.
- ☐ El destino está registrado en el array de NPCDestinationManager.
- ☐ El número de slots refleja la capacidad deseada; CurrentOccupants cuenta reservas activas.

8.3. Lista de comprobación al añadir una conexión

- ☐ El GameObject tiene NPCFloorConnection.
- ☐ Floor A / Floor B forman una conexión válida entre plantas adyacentes.
- ☐ Point A y Point B están asignados y colocados sobre sus respectivas superficies NavMesh.
- ☐ Bidirectional está activado para las escaleras normales.
- ☐ La conexión está incluida en Available Connections de cada NPC que pueda utilizarla.
- ☐ La ruta funciona en ambos sentidos y en viajes de varias plantas.

8.4. Limitaciones y detalles conocidos de la implementación actual

- CanReachDestination(). Comprueba destination.Position (el centro), no un slot libre. MoveToDestination() sí valida el slot reservado. Si la IA usa la prevalidación con destinos con slots, conviene alinearla con el slot concreto.
- Fallos durante el movimiento. NPCNavigation detecta una ruta incompleta, pero no avisa actualmente a NPCFloorNavigator mediante un evento. Algunos fallos de tramos intermedios llaman a CancelCurrentTrip() y se notifican como cancelación. Unificarlo antes de implementar reintentos automáticos.
- Ocupación. CurrentOccupants cuenta reservas desde que se asigna el slot, también durante el trayecto. Los slots no sustituyen la evitación local del NavMeshAgent.
- WebGL. Validar una compilación WebGL real; que funcione en el Editor no garantiza el resultado en navegador.

8.5. Qué no hace el sistema de navegación

La navegación no decide la actividad, su duración ni las reacciones del estudiante: eso corresponde a la IA y al gameplay. La API de estados debe comunicarse con NPCDestinationManager y NPCFloorNavigator; no debe controlar NavMeshAgent ni mover el Transform directamente. Al terminar una actividad, debe liberar el slot; al interrumpir un viaje, debe cancelarlo.